

Building a wiki farm is a step up from a standard CMS because you're essentially playing "God" over multiple independent instances. Since you're developing the **ARC Panel (Listiary ARC)**, you're likely moving toward a centralized management approach where the installer isn't just a setup wizard, but a provisioning engine.

Here is a breakdown of how the big players (like MediaWiki, WordPress Multisite, or DokuWiki) handle their installers and what actually happens under the hood.

1. What is actually downloaded?

In most PHP ecosystems, there are two main "flavors" of what the user starts with:

- **The Full Package:** A .zip or .tar.gz containing the entire codebase (e.g., MediaWiki core). The user unzips this into their web directory.
- **The Bootstrapper (Web Installer):** A single install.php file. When run, this file uses PHP's cURL or file_get_contents to fetch the latest version of the core system from a remote server, unzips it locally, and then starts the setup.

2. The Setup Wizard: What runs where?

The installer is almost always a web-based GUI. It's a series of PHP scripts that run in the user's browser.

Phase	What Happens	Why it matters
Environment Check	The script checks php.ini settings (memory limit, upload size) and required extensions (PDO, GD, mbstring, cURL).	Prevents "White Screen of Death" after installation.
Permission Audit	Checks if /config, /images, or /cache folders are writable.	PHP needs to be able to "write" the configuration file it's about to create.
Database Handshake	User enters DB credentials. PHP attempts to connect and checks permissions (CREATE,	Ensures the script can actually build the architecture.

	DROP, ALTER).	
--	---------------	--

3. Beyond Tables: What the installer *actually* does

You mentioned you know they create tables, but for a **Wiki Farm**, the installer does several more "invisible" tasks:

A. Generating the Unique Salt/Secret Key

The installer generates a long, random string (a "salt") stored in the configuration file. This is used to encrypt user cookies and passwords. In a farm, each wiki needs its own unique salt so a login on Wiki A doesn't accidentally grant access to Wiki B.

B. The "Birth" of the Config File

This is the most critical step. The installer takes the user's input and writes a file (like LocalSettings.php or config.inc.php).

- **Standard Wiki:** Writes the DB credentials directly.
- **Wiki Farm:** The "Master Installer" (your ARC Panel) might create a directory for the new wiki and place a "pointer" file there, or update a master "Farm Map" database that tells the core code which DB belongs to which URL.

C. Directory Provisioning

In a farm, the installer usually creates a specific folder for that sub-wiki to store its own:

- Uploaded images/files.
- Custom CSS/Skins.
- Local cache.

4. How Wiki Farms specifically handle "Installation"

Since your **ARC Panel** is a separate tool, your installer likely works in one of two ways common to farms:

The "Symlink" Method

The code lives in one folder (/core). When you "install" a new wiki via the ARC Panel, the panel creates a new folder (/wiki1), and inside it, it creates **Symbolic Links** to the core files. It only places a unique local_config.php in that folder.

- *Pros:* Updating the core code once updates the whole farm.
- *Cons:* Can be tricky with some shared hosting permissions.

The "Virtual" Method (Routing)

There is only one set of files and one index.php. When a request comes in for alpha.farm.com, the PHP code looks at the `$_SERVER['HTTP_HOST']`, checks your ARC Panel database, and says: *"Aha! This is Wiki #42. Use 'db_alpha' and load these settings."*

- *In this case:* The "installer" in your ARC Panel isn't moving files; it's just inserting a new row into your Master Database and running the SQL schema on a new database.

Summary of the Workflow

1. **Request:** ARC Panel receives a "Create New Wiki" command.
2. **Validation:** Panel checks if the name/URL is available.
3. **Provisioning:** Panel creates a new database and a new filesystem directory (or just a DB entry).
4. **Schema Injection:** Panel runs a massive .sql file to populate the new DB with tables and "Seed Data" (like the default "Main Page" and the Admin user).
5. **Finalization:** Panel writes the connection details to a protected config file that the wiki's "Entry Point" can read.

A Quick Tip: If you're building this for others to use, always include a **"Connection Test"** button before the installer actually tries to write to the database. It saves users from a world of frustration when they inevitably typo their DB password.

How are you planning to handle the "farm" logic—will each wiki have its own folder, or are you doing it all through URL routing?